



AWS Academy Cloud Developing
Module 12 Student Guide
Version 2.0.6

200-ACCDEV-20-EN-SG

© 2024, Amazon Web Services, Inc. or its affiliates. All rights reserved.

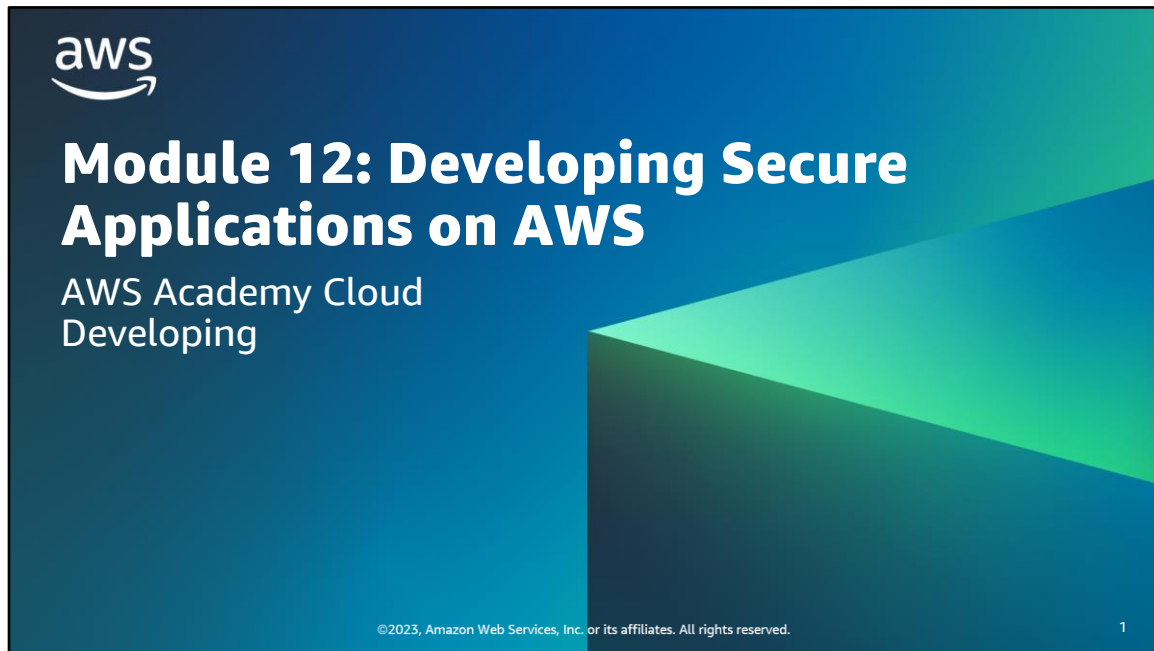
This work may not be reproduced or redistributed, in whole or in part,
without prior written permission from Amazon Web Services, Inc.
Commercial copying, lending, or selling is prohibited.

All trademarks are the property of their owners.

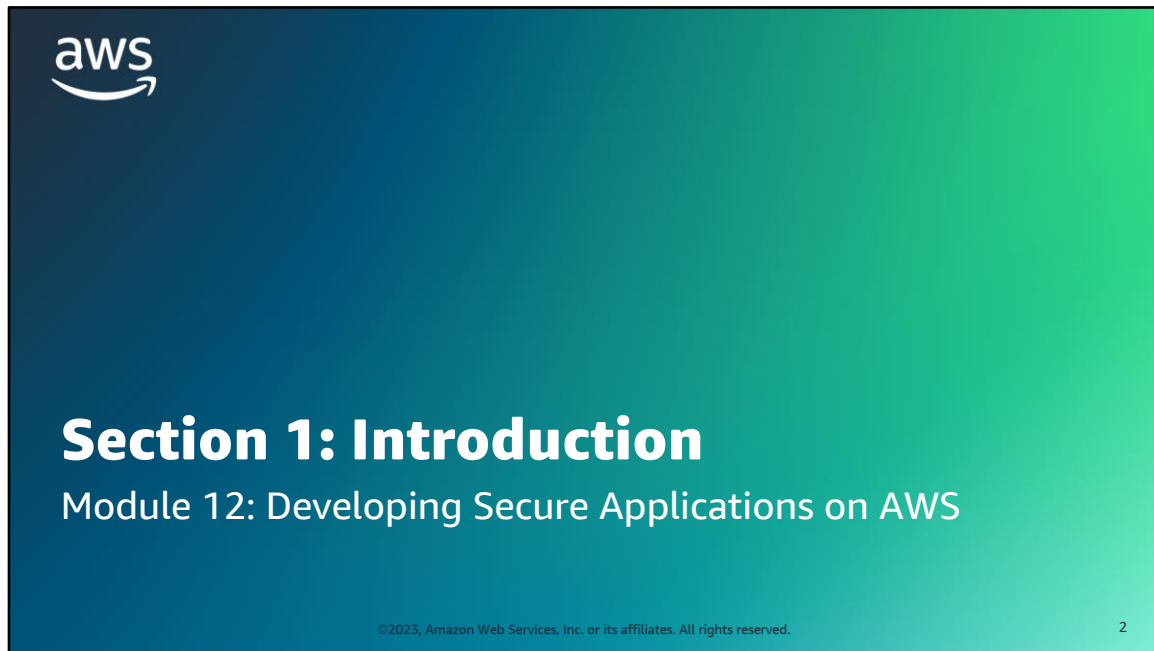
Contents

[Module 12: Developing Secure Applications on AWS](#)

4



Welcome to Module 12: Developing Secure Applications on AWS.



Section 1: Introduction.

Module objectives

At the end of this module, you should be able to do the following:

- List two aspects of securing application
- Recall how to authenticate with the AWS Security Token Service (AWS STS)
- Describe how to build secure applications with Amazon Cognito
- Secure part of a website with Amazon Cognito



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

3

At the end of this module, you should be able to do the following:

- List two aspects of securing applications
- Recall how to authenticate with the AWS Security Token Service (AWS STS)
- Describe how to build secure applications with Amazon Cognito
- Secure part of a website with Amazon Cognito

Module overview

Sections

1. Introduction
2. Securing network connections
3. Authenticating with AWS STS
4. Authenticating with Amazon Cognito

Lab

- Implementing Application Authentication Using Amazon Cognito



Knowledge check



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

4

This module includes the following sections:

1. Introduction
2. Securing network connections
3. Authenticating with AWS STS
4. Authenticating with Amazon Cognito

This module also includes a lab on Implementing Application Authentication Using Amazon Cognito.

Finally, you will complete a knowledge check to test your understanding of key concepts covered in this module.

Café business requirement

Sofía wants to modify the café website to provide a login for café employees that gives them access to features that aren't available to external users.



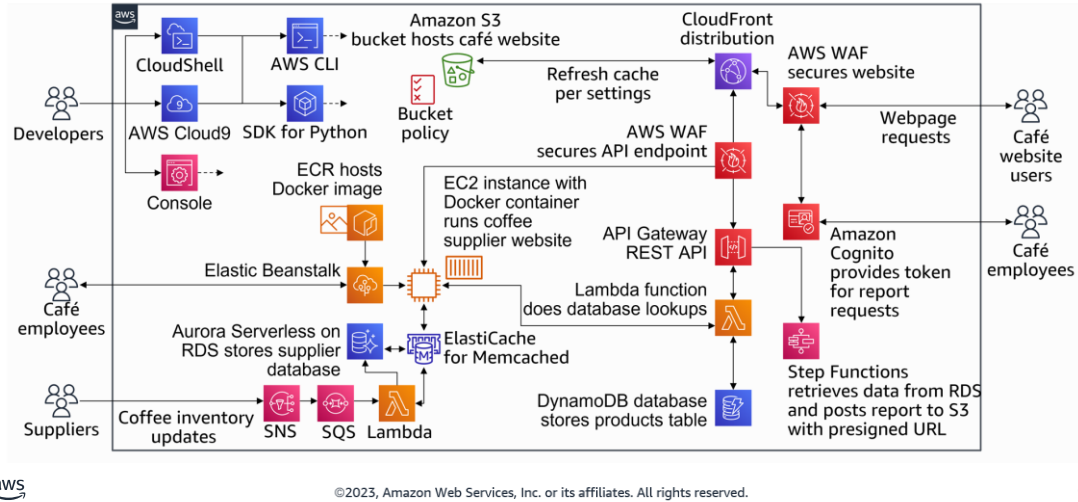
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

5

Sofía wants to modify the café website to provide a login for café employees that gives them access to features that aren't available to external users.

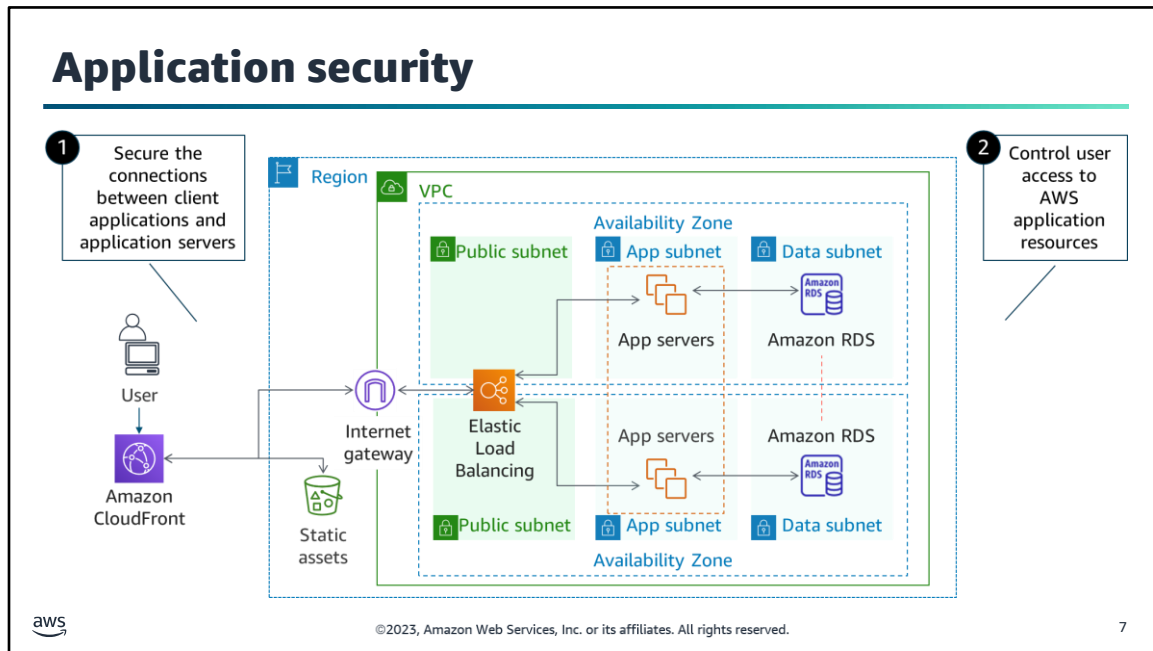
Sofía doesn't have much experience with authentication and authorization. Faythe, an AWS developer and café regular, suggests using Amazon Cognito. The service is easy to set up and integrate with the AWS services that the café website uses.

Amazon Cognito as part of developing a cloud application



The diagram on this slide gives an overview of the application that you will build through the labs in this course. The highlighted portions are relevant to this module.

As highlighted in the diagram, you will use Amazon Cognito to provide a login for café employees that gives them access to features that aren't available to external users.



This example AWS architecture illustrates the layers of a web-based application. Recall that with the AWS shared responsibility model, AWS is responsible for the security **of** the cloud, and you are responsible for security **in** the cloud. Developers must consider security in the cloud at all layers of an application.

You have previously learned about two services that are core to AWS security:

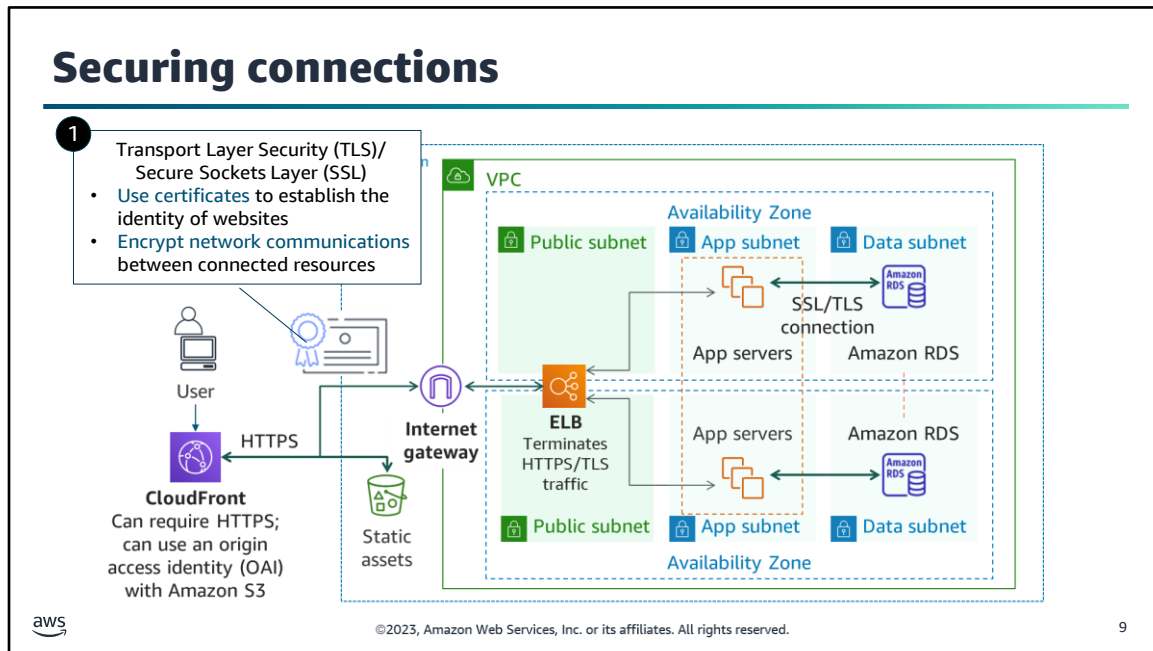
- With Amazon Virtual Private Cloud (Amazon VPC), you secure your networks and subnets, and protect your AWS resources in the cloud.
- With AWS Identity and Access Management (IAM), you control access to AWS resources through policies that grant permissions to users, groups, and roles.

In this module, you'll learn about two aspects of applying security to your application:

- Securing the network connections between application clients and application servers over the internet
- Controlling user access to AWS resources called by your application



Section 2: Securing network connections.



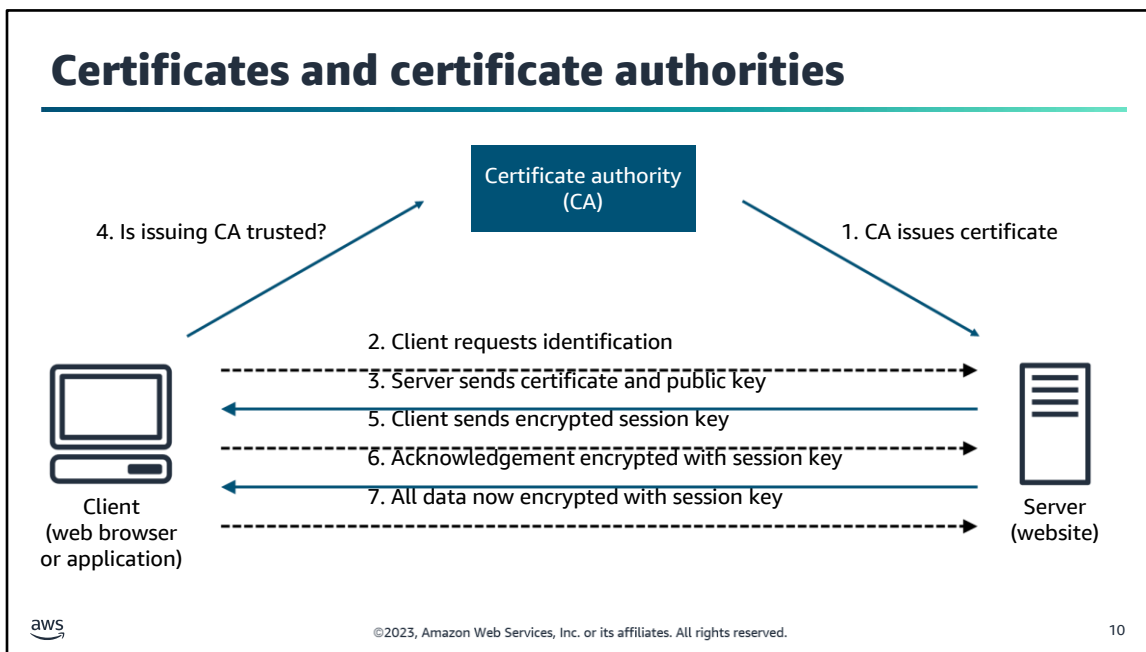
Transport Layer Security (TLS) and its predecessor Secure Sockets Layer (SSL) are open standards that use public and private *certificates* to establish the identity of websites over the internet and resources on private networks. SSL and TLS protocols also encrypt network communications between connected resources. For example, Amazon.com uses TLS for all traffic on its website, and AWS uses TLS to secure API calls to AWS services.

HTTPS connections might use either SSL or TLS. Although the term SSL is still widely used, TLS is typically being used to secure the connection.

In this example, Elastic Load Balancing simplifies the process of building secure web applications by terminating HTTPS and TLS traffic from clients at the load balancer. The load balancer encrypts and decrypts the traffic, instead of requiring each Amazon Elastic Compute Cloud (Amazon EC2) instance to handle the work for TLS termination.

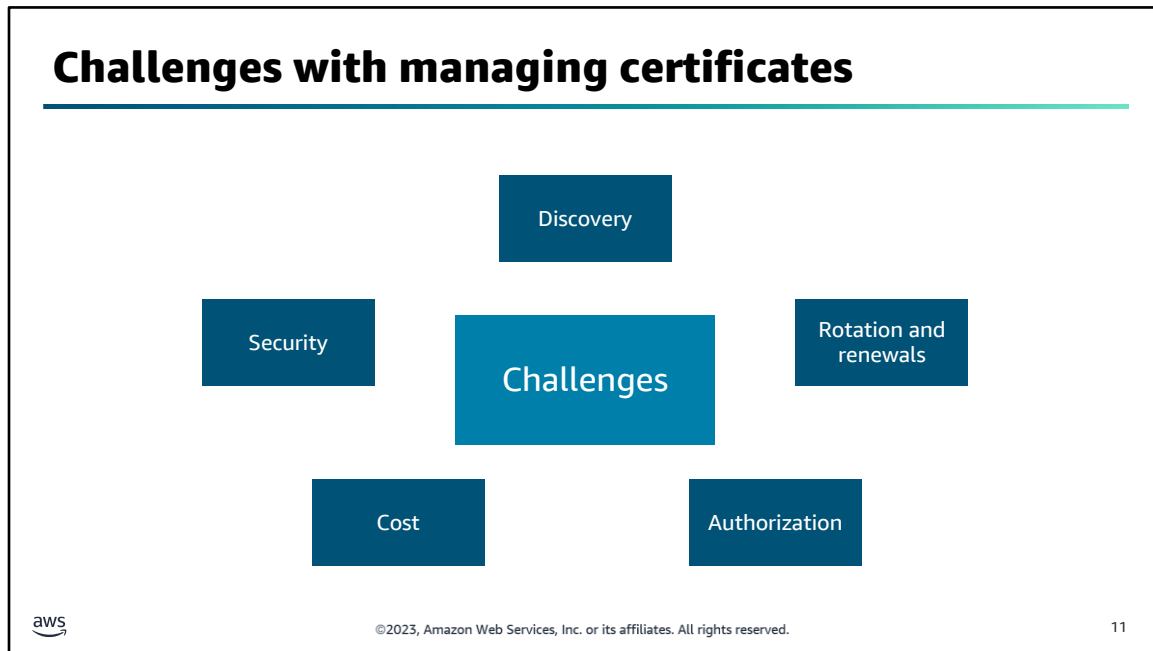
You can configure Amazon CloudFront to require that viewers use HTTPS so that connections are encrypted when CloudFront communicates with viewers. You also can configure CloudFront to use HTTPS with your origin so that connections are encrypted when CloudFront communicates with your origin. CloudFront performs SSL/TLS negotiation between the viewer and CloudFront, and between CloudFront and the origin if the response is not already cached.

In this example, you might set up an origin access identify (OAI) in CloudFront with Amazon Simple Storage Service (Amazon S3) bucket permissions so that only CloudFront can access the bucket through the OAI. This means that users cannot directly access content on the S3 bucket even if they have the URL.



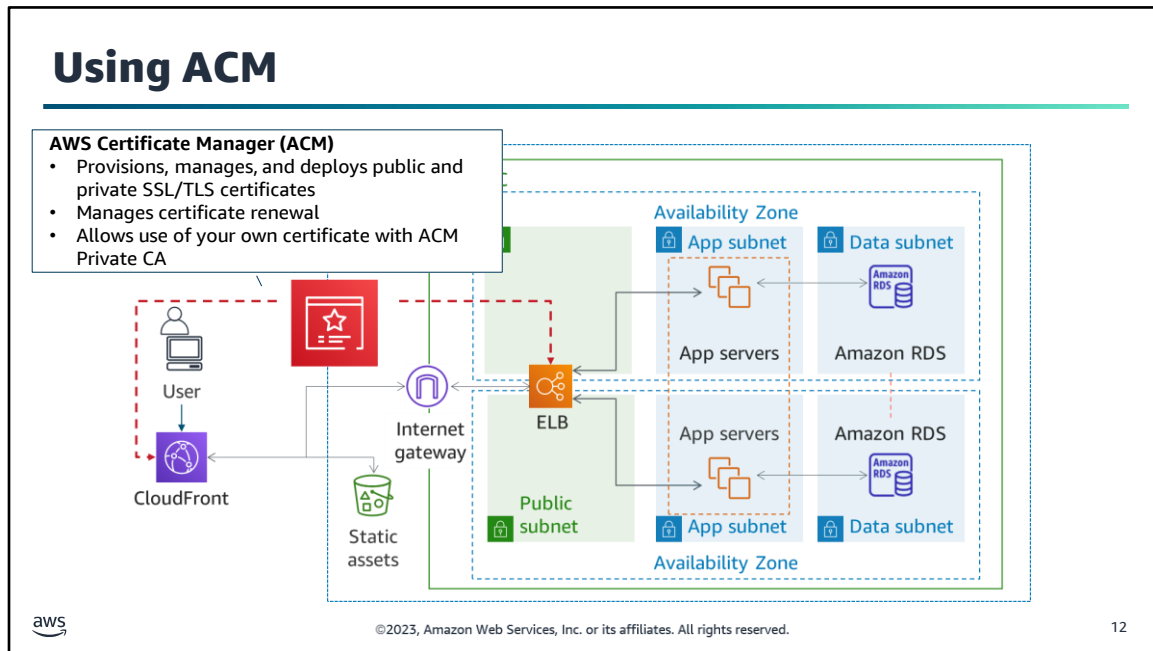
Certificate authorities, also known as *CAs*, issue certificates to specific domains. When a domain presents a certificate that a trusted CA issued, your browser or application knows it's safe to make the connection.

The steps listed in the slide show how the TLS and SSL protocols use the certificate to ensure that the client connecting is a trusted resource.



Managing your SSL/TLS certificates presents several challenges:

- **Security:** Certificates can be vulnerable for many reasons (for example, certificate name mismatch, use of internal names, missing or misconfigured fields and values, outdated or weak hashing algorithms, weak keys, and weak cypher suites that compromise SSL endpoints). It can be challenging to find and fix these vulnerabilities.
- **Discovery:** It is not practical to manually gather details about all of the individual certificates in your network when you have hundreds or thousands of them.
- **Rotations and renewals:** Manual management of certificate expirations is prone to error, and could cause you to miss rotations and renewals.
- **Authorization:** You need to be able to verify that someone is authorized to approve and issue a certificate.
- **Cost:** It can be expensive to manage certificates. You must pay a fee to validate certificates. You must also consider support costs, legal costs (including insurance, warranties, and so forth), ownership or control costs (such as fees for root-embedding), and infrastructure costs.



With the AWS Certificate Manager (ACM) service, you can provision, manage, and deploy public and private SSL/TLS certificates for use with AWS services and your internal connected resources. ACM removes the time-consuming, manual process to purchase, upload, and renew SSL/TLS certificates.

With ACM, you can quickly request a certificate and deploy it on AWS resources that are integrated with ACM. These resources include ELB load balancers, CloudFront distributions, and APIs on Amazon API Gateway. The service manages certificate renewals. You can also create private certificates for your internal resources and centrally manage the certificate lifecycle. Public and private certificates that you provision through ACM for use with AWS services are free. You pay only for the AWS resources that you create to run your application.

You can also produce your own certificate with AWS Certificate Manager Private Certificate Authority.

ACM makes it easy to enable SSL/TLS, which helps organizations to meet regulatory and compliance requirements for encryption of data in transit.

For more information, see the following resources:

- AWS Certificate Manager: <https://aws.amazon.com/certificate-manager/>.
- AWS Certificate Manager Private Certificate Authority: <https://aws.amazon.com/certificate-manager/private-certificate-authority/>.
- Amazon Trust Services: <https://www.amazontrust.com/repository/>.

This slide illustrates the use of ACM to act as the CA to generate certificates for both CloudFront and ELB.

Section 2 key takeaways



- You can encrypt **network communications** with **SSL** and **TLS** protocols.
- SSL and TLS require a **certificate** issued by a **CA**.
- **ACM** simplifies **management** and **renewal** of certificates.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

13

The following are the key takeaways from this section of the module:

- You can encrypt network communications with SSL and TLS protocols.
- SSL and TLS require a certificate issued by a CA.
- ACM simplifies management and renewal of certificates.



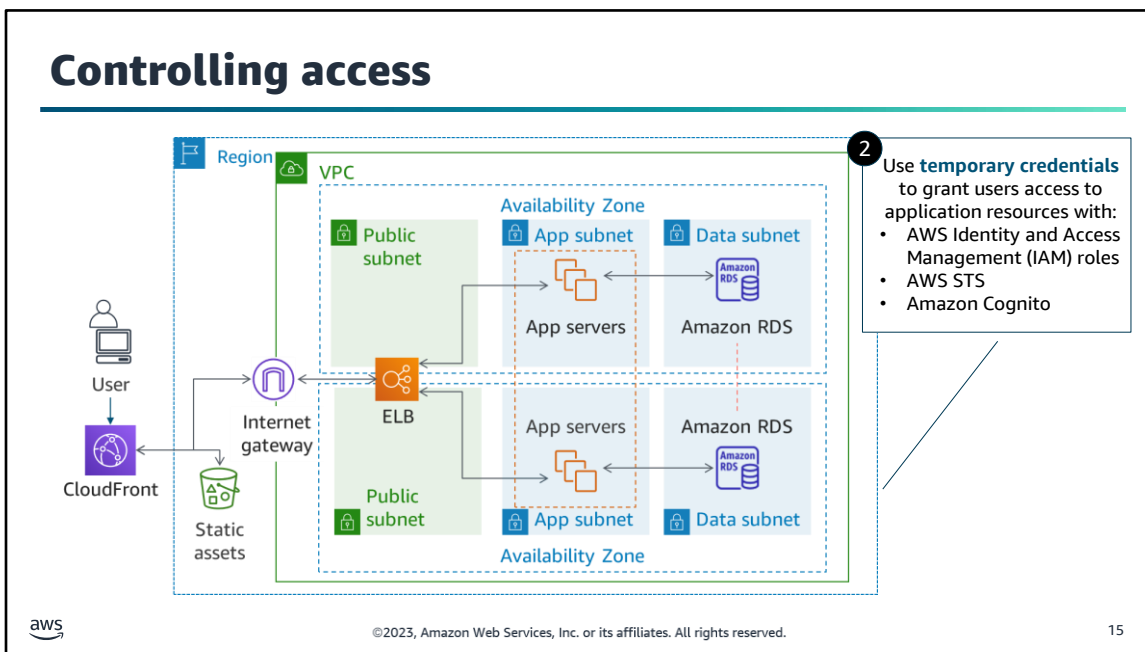
Section 3: Authenticating with AWS STS

Module 12: Developing Secure Applications on AWS

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

14

Section 3: Authenticating with AWS STS.



As noted earlier in the module, the second aspect of security that you'll learn about is controlling application users' access to your application resources.

In the next two sections, you learn about using the AWS Security Token Service (AWS STS) and Amazon Cognito in conjunction with AWS Identity and Access Management (IAM) to control access to resources.

As noted on the slide, it is a security best practice to use temporary credentials to grant permissions to your resource.

Key concepts for access control

- **Authentication:** Verify the identity of the user
- **Authorization:** Verify the user's permissions (what the user is allowed to do)
- **Identity provider (IdP):** Manages identity information and provides authentication services
- **Identity broker:** Authenticates credentials against an IdP and retrieves temporary security credentials from AWS STS
- **Standards:**
 - **Security Assertion Markup Language (SAML):** Open standard used to exchange authentication and authorization data between parties
 - **OpenID Connect (OIDC):** Open standard that third-party IdPs use so that other companies/sites can use them to authenticate users without having to maintain an in-house user database
 - **JSON Web Token (JWT):** Open standard used to securely transmit information between two parties



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

16

To understand how AWS access control works, familiarize yourself with the following key concepts:

- **Authentication:** Verifies the user's identity
- **Authorization:** Verifies the user's permissions (or what the user is allowed to do)
- **Identity provider (IdP):** Manages identity information and provides authentication services
- **Identity broker:** Software layer that authenticates credentials against an IdP and retrieves temporary security credentials from AWS STS
- **Standards:**
 - **Security Assertion Markup Language (SAML):** Open standard used to exchange authentication and authorization data between parties
 - **OpenID Connect (OIDC):** Open standard that third-party IdPs use so that other companies or sites can use them to authenticate users without having to maintain an in-house user database
 - **JSON Web Token (JWT):** Open standard used to securely transmit information between two parties

Why use temporary security credentials



Bad practice

- Embed access keys in unencrypted code.
- Share access keys between users in an AWS account.



Best practice

- Use IAM roles to retrieve temporary security credentials.
- If IAM roles are not an option, use AWS STS to retrieve credentials.



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

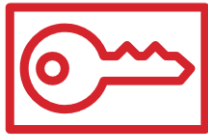
17

As you learned in the IAM module, you can authenticate programmatically to AWS services through the AWS Command Line Interface (AWS CLI), SDKs, and APIs by using your AWS access key. The *access key* is a combination of your access key ID and secret access key.

Embedding access keys in unencrypted code and sharing security credentials between users in your AWS account are bad security practices. If your application or application users need to access AWS services, you should configure *temporary security credentials*.

For more information about best practices for temporary security credentials, see Security Best Practices in IAM at <https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html>.

Temporary security credentials from AWS STS



AWS Security
Token Service
(AWS STS)

- Provided to trusted users to enable them to access your AWS resources
- Short-lived credentials that consist of an access key ID, secret access key, and session token
- Limited in duration to a configurable lifetime
- Not reusable after they expire
- Generated dynamically



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

18

You can use the AWS Security Token Service (AWS STS) to provide trusted users with temporary security credentials to access your AWS resources. Temporary security credentials consist of a short-lived access key ID, secret access key, and session token.

As the name implies, temporary security credentials have a limited lifetime. You can configure them to last from a few minutes to several hours. After the credentials expire, AWS no longer recognizes them or allows any kind of access from API requests that are made with them.

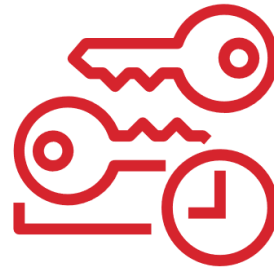
Temporary security credentials are not stored with the user but are generated dynamically and provided to the user when requested. When (or even before) the temporary security credentials expire, the user can request new credentials, as long as the user who requests them still has permissions to do so.

For more information, see the following resources:

- AWS STS API Reference:
<https://docs.aws.amazon.com/STS/latest/APIReference/Welcome.html>.
- Temporary Security Credentials in IAM:
https://docs.aws.amazon.com/IAM/latest/UserGuide/id_credentials_temp.html.
- AWS Services That Work with IAM:
https://docs.aws.amazon.com/IAM/latest/UserGuide/reference_aws-services-that-wrk-with-iam.html.

AWS STS trusted users

- IAM users
 - For cross-account access
 - For applications running on Amazon Elastic Compute Cloud (Amazon EC2) instances and other AWS compute services
- Federated identities for external users
 - Enterprise (single sign-on)
 - Web (social sign-in)



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

19

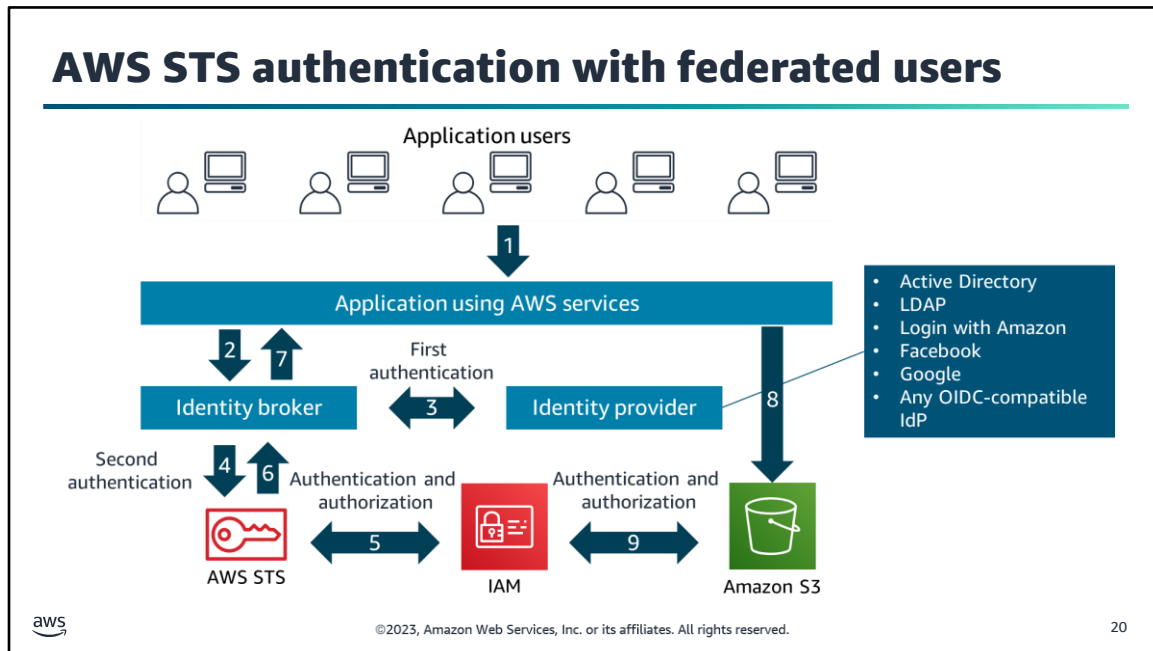
AWS STS trusted users can be IAM users or federated identities:

- **IAM users:** You can establish cross-account access for IAM users in one AWS account who need temporary access to AWS resources in another AWS account. To do this, you can provide temporary security credentials to an IAM role, which the IAM user assumes. You can also use IAM roles with temporary security credentials to manage access to applications that are running in an Amazon EC2 environment and other AWS compute services.
- **Federated identities:** *Federated identities* are users who sign in to your application from an authentication system outside of AWS. IAM supports two types of identity federation:
 - **Enterprise identity federation:** For employees who are on a corporate network and are authenticated through a SAML 2.0-compatible enterprise IdP, such as Active Directory or LDAP.
 - **Web identity federation:** For mobile and web-based application users who are authenticated through an online, third-party IdP, such as Login with Amazon, Facebook, Google, or any OIDC-compatible IdP.

For federated identities, you do not need to create new AWS identities for users, and require them to sign in to your application with a separate user name and password. Instead, users can access your AWS resources directly by using their corporate network credentials (referred to as *single sign-on*, or *SSO*) or a third party, such as Login with Amazon, Facebook, or Google (referred to as *social sign-in*).

For more information, see the following resources:

- Identity Providers and Federation:
https://docs.aws.amazon.com/IAM/latest/UserGuide/id_roles_providers.html.
- Federating Users by Creating a Custom Identity Broker Application:
https://docs.aws.amazon.com/IAM/latest/UserGuide/id_roles_common-scenarios_federated-users.html#id_roles_common-scenarios_federated-users-idbroker.



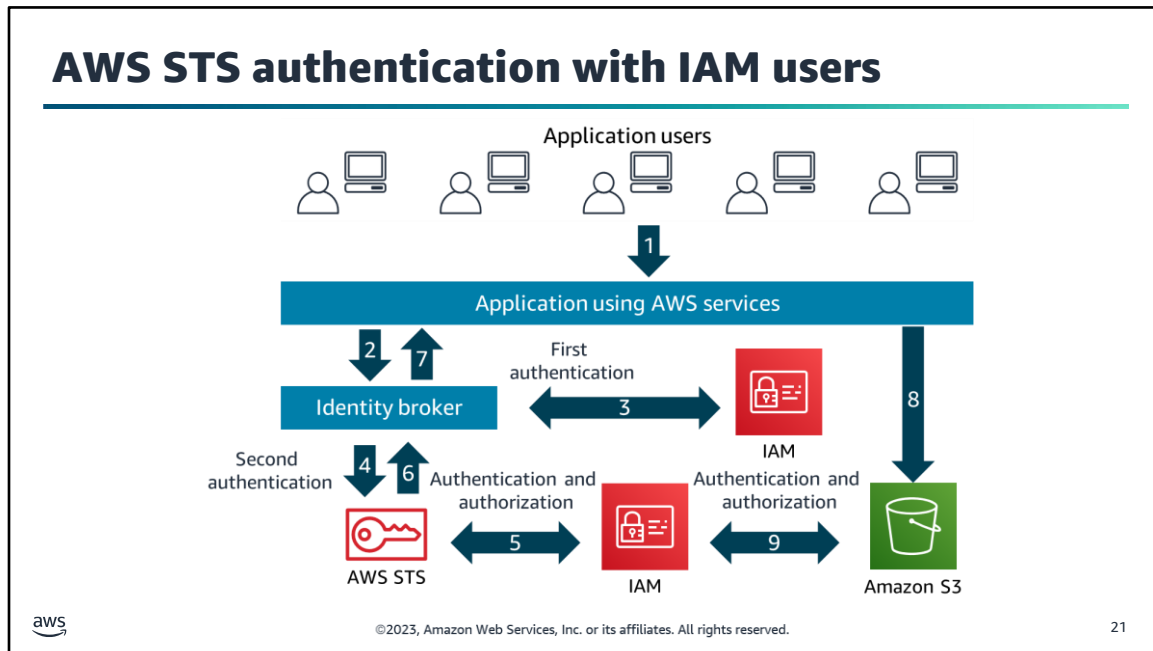
The following is an explanation of how authentication with AWS STS works for federated users:

1. A user accesses an application that is backed by AWS.
2. The application calls an *identity broker*. The identity broker accepts a user's identifier as input.
3. (First authentication) The identity broker first authenticates the user's identity against an *identity provider* (IdP), such as Active Directory (for enterprise federation) or an online, third-party IdP (for web federation).
4. (Second authentication) If the authentication is successful, the identity broker makes an API call to AWS STS. The call must include an IAM policy and a duration, along with a policy that specifies the permissions to be granted to the temporary security credentials.
5. AWS STS uses IAM to confirm that the policy of the IAM user who is making the API call has permissions to create new tokens.
6. AWS STS returns four values to the identity broker: an access key, secret access key, session token, and duration (that is, the token's lifetime).
7. The identity broker returns the temporary security credentials and token to the application.
8. The application uses the temporary security credentials and token to make requests to an AWS service, such as Amazon S3.
9. The AWS service uses IAM to confirm that the credentials allow the requested operation on the given resource.

For more information about how authentication with AWS STS works, see the following resources:

- AWS Identity and Access Management – Now with Identity Federation:
<https://aws.amazon.com/blogs/aws/aws-identity-and-access-management-now-with-identity-federation/>.
- Requesting Temporary Security Credentials:
https://docs.aws.amazon.com/IAM/latest/UserGuide/id_credentials_temp_reques

t.html.



The following is an explanation of how authentication with AWS STS works with IAM users:

1. A user accesses an application that is backed by AWS.
2. The application calls an *identity broker*. The identity broker accepts a user's identifier as input.
3. (First authentication) The identity broker first authenticates the user's identity by checking whether they are an IAM user.
4. (Second authentication) If the authentication is successful, the identity broker makes an API call to AWS STS. The call must include an IAM policy and a duration, along with a policy that specifies the permissions to be granted to the temporary security credentials.
5. AWS STS uses IAM to confirm that the policy of the IAM user who is making the API call has permissions to create new tokens.
6. AWS STS returns four values to the identity broker: an access key, secret access key, session token, and duration (that is, the token's lifetime).
7. The identity broker returns the temporary security credentials and token to the application.
8. The application uses the temporary security credentials and token to make requests to an AWS service, such as Amazon S3.
9. The AWS service uses IAM to confirm that the credentials allow the requested operation on the given resource.

For more information about how authentication with AWS STS works, see the following resources:

- AWS Identity and Access Management – Now with Identity Federation:
<https://aws.amazon.com/blogs/aws/aws-identity-and-access-management-now-with-identity-federation/>.
- Requesting Temporary Security Credentials:
https://docs.aws.amazon.com/IAM/latest/UserGuide/id_credentials_temp_request.html.

AWS STS API operations

Operation	Returns Set of Temporary Security Credentials to...
<code>AssumeRole</code>	Existing IAM users for cross-account access to AWS resources.
<code>AssumeRoleWithSAML</code>	Federated users who are authenticated by an organization's existing identity system.
<code>AssumeRoleWithWebIdentity</code>	Federated users who are authenticated through a public IdP.
<code>GetFederationToken</code>	Federated users. This API differs from <code>AssumeRole</code> in that the default expiration period is substantially longer (12 hours instead of 1 hour).
<code>GetSessionToken</code>	Existing IAM users for enhanced security, such as allowing AWS requests only when multi-factor authentication (MFA) is enabled for the IAM user.



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

22

Different AWS STS API operations return temporary security credentials:

- **AssumeRole:** Returns a set of temporary security credentials for existing IAM users to grant cross-account access to AWS resources.
- **AssumeRoleWithSAML:** Returns a set of temporary security credentials for federated users who are authenticated by an organization's existing identity system. The users must also use SAML 2.0 to pass authentication and authorization information to AWS.
- **AssumeRoleWithWebIdentity:** Returns a set of temporary security credentials for federated users who are authenticated through a public IdP, such as Login with Amazon, Facebook, Google, or any OIDC-compatible identity provider. This API is useful to create mobile applications or client-based web applications that require access to AWS in which users do not have their own AWS or IAM identities. However, instead of directly calling `AssumeRoleWithWebIdentity`, we recommend that you use Amazon Cognito and the Amazon Cognito credentials provider with the AWS SDKs for mobile development.
- **GetFederationToken:** Returns a set of temporary security credentials for federated users. This API differs from `AssumeRole` in that the default expiration period is substantially longer (12 hours instead of 1 hour).
- **GetSessionToken:** Returns a set of temporary security credentials for an existing IAM user. This is useful to provide enhanced security, such as allowing AWS requests only when multi-factor authentication (MFA) is enabled for the IAM user.

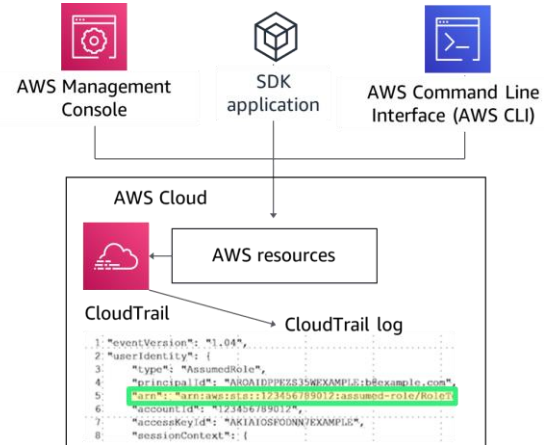
For more information, see the following resources:

- AWS STS API operations:
<https://docs.aws.amazon.com/STS/latest/APIReference/Welcome.html>.
- Comparing the AWS STS API Operations:
https://docs.aws.amazon.com/IAM/latest/UserGuide/id_credentials_temp_request.html#stsapi_comparison.

Tracking user actions with CloudTrail

CloudTrail captures actions from authenticated users

- Stores log files in Amazon S3
- Can be configured to respond with actions or notifications



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

23

You can use AWS CloudTrail to track the activity of federated users; for example, a SAML-federated user who terminated an EC2 instance in your account, or a mobile application user who signed into your application with their Facebook account and deleted a photo from your S3 bucket. The ability to track federated users can help you to conduct audits of their activities, which can help you with your compliance and security efforts. To capture the activity of federated users, CloudTrail records the AssumeRoleWithSAML and AssumeRoleWithWebIdentity AWS STS API calls.

CloudTrail: Track federated user actions

```

1  "eventVersion": "1.04",
2  "userIdentity": {
3    "type": "AssumedRole",
4    "principalId": "AROAI0PPEZS35WEXAMPLE:b@example.com",
5    "arn": "arn:aws:sts::123456789012:assumed-role/RoleToBeAssumed/b@example.com",
6    "accountId": "123456789012",
7    "accessKeyId": "AKIAIOSFODNN7EXAMPLE",
8    "sessionContext": {
9      "attributes": {
10       "creationDate": "20131102T010628Z",
11       "mfaAuthenticated": "false"
12     },
13     "sessionIssuer": {
14       "type": "Role",
15       "principalId": "AROAI0PPEZS35WEXAMPLE",
16       "arn": "arn:aws:iam::123456789012:role/RoleToBeAssumed",
17       "accountId": "123456789012",
18       "userName": "RoleToBeAssumed"
19     }
20   },
21 },
22 "eventTime": "2016-03-09T21:50:16Z",
23 "eventSource": "ec2.amazonaws.com",
24 "eventName": "TerminateInstances",
25 "awsRegion": "us-east-1",
26 "sourceIPAddress": "127.0.0.1",
27 "userAgent": "signin.amazonaws.com",
28 "errorCode": null,
29 "errorMessage": null

```

1. Search for eventName
TerminateInstances

2. Determine the Amazon
Resource Name (ARN) of
the IAM role that the
federated user assumed



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

24

To understand how you can use CloudTrail to capture the activity of federated users, consider this example. The Example.com organization has an IAM administrator named Alice and an employee named Bob. Example.com has configured its SAML 2.0-compliant IdP and AWS to permit federated users such as Bob (email address: b@example.com) to access the AWS Management Console. Bob signs into the console through SSO by using SAML 2.0, and he terminates an EC2 instance. Alice learns that a user has terminated the EC2 instance. She uses CloudTrail to identify which federated user terminated the instance.

First, Alice searches the CloudTrail event logs for the eventName called *TerminateInstances*. In the *userIdentity* section of the event log, Alice determines the Amazon Resource Name (ARN) of the IAM role that the federated user assumed.

CloudTrail: Track federated user actions (continued)

```

1 {
2   "eventVersion": "1.04",
3   "userIdentity": {
4     "type": "SAMLUser",
5     "principalId": "71EC1xd9IdqExample:Bob",
6     "userName": "Bob",
7     "identityProvider": "71EC1xd9IdqExample"
8   },
9   "eventTime": "2016-03-09T01:22:27Z",
10  "eventSource": "sts.amazonaws.com",
11  "eventName": "AssumeRoleWithSAML",
12  "sourceIPAddress": "127.0.0.1",
13  "userAgent": "signin.amazonaws.com",
14  "requestParameters": {
15    "sAMLAssertionID": "c0046ce598b94b9d4b8e45027Example",
16    "roleSessionName": "b@example.com",
17    "durationSeconds": 3600,
18    "roleArn": "arn:aws:iam:123456789012:role/RoleToBeAssumed",
19    "principalArn": "arn:aws:iam:123456789012:saml-provider/Shibboleth"
20  },
21  "responseElements": {
22    "subjectType": "transient",
23    "issuer": "https://sts-integ.amazon.com/idp/shibboleth",
24    "credentials": {
25      "accessKeyId": "ASIAIDMUEK632EXAMPLE",
26      "expiration": "Mar 9, 2016 2:22:27 AM",
27      "sessionToken": "encoded session token blob"
28    },
29    "nameQualifier": "71EC1xd9IdqExample",
30    "assumedRoleUser": {
31      "assumedRoleId": "AROAIIDPPEZS35EXAMPLE:RoleToBeAssumed",
32      "arn": "arn:aws:sts:123456789012:assumed-role/RoleToBeAssumed/b@example.com"
33    }
34  }
35 }

```

4. Identify the SAML-federated user identity in the userName attribute

3. Search the CloudTrail event logs for the eventName called AssumeRoleWithSAML and the IAM role's ARN



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

25

Alice then searches the CloudTrail event logs for the eventName called *AssumeRoleWithSAML* that includes the IAM role's ARN. Finally, Alice identifies Bob as the federated user in the userName attribute of the userIdentity section of the CloudTrail event log.

For detailed instructions and links to additional resources, see the [How to Easily Identify Your Federated Users by Using AWS CloudTrail](https://aws.amazon.com/blogs/security/how-to-easily-identify-your-federated-users-by-using-aws-cloudtrail/) blog post at <https://aws.amazon.com/blogs/security/how-to-easily-identify-your-federated-users-by-using-aws-cloudtrail/>.

Section 3 key takeaways



- A **best practice** is to issue temporary credentials to access AWS resources.
- **AWS STS** provides temporary credentials for IAM users and external users.
- Use **CloudTrail** to track actions from federated (external) users.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

26

The following are the key takeaways from this section of the module:

- A best practice is to issue temporary credentials to access AWS resources.
- AWS STS provides temporary credentials for IAM users and external users.
- Use CloudTrail to track actions from federated (external) users.



Section 4: Authenticating with Amazon Cognito.

Amazon Cognito



Amazon Cognito

Provides **authentication**, **authorization**, and **user management** for your web and mobile applications

- Authenticates user identities through external IdPs that support SAML or OIDC, social IdPs, and custom IdPs
- Provides temporary security credentials to access AWS resources and services



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

28

The Amazon Cognito service provides authentication, authorization, and user management for your web and mobile applications.

You can use Amazon Cognito to do the following:

- Create unique identities for your users and authenticate the identities with IdPs. Amazon Cognito works with external IdPs that support SAML or OIDC, social IdPs (such as Facebook, Twitter, and Amazon), and your own custom IdPs.
- Provide temporary security credentials to your application to access AWS resources and services.

Amazon Cognito user pools and identity pools (federated identities)

User pools

- Directory of users
- Provide sign-in for users
- Accept federated identities from Login with Amazon, Google, Facebook, or any SAML or OIDC IdPs

Identity pools (federated identities)

- Provide users with temporary access credentials
- Enable creation of unique identities and federate them with various providers (for example, an OIDC identity or Amazon identity)



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

29

The two main components of Amazon Cognito are user pools and identity pools:

- *User pools* are user directories that provide sign-up and sign-in options for your application users. Users can sign in to your web or mobile application through Amazon Cognito or federate through a third-party IdP. Whether users sign in directly or through a third party, all members of the user pool have a directory profile, which you can access through an SDK.
- *Identity pools* (also called federated identities) provide your users with temporary security credentials to access other AWS services.

You can have multiple identities for various IdPs. For example, you can set up an identity for OIDC login and another identity for Apple users to log in.

As you will see, you can use identity pools and user pools separately or together.

User pool security features and functionality

- Sign-up and sign-in services
- Built-in, customizable login web UI
- Control who can access your API
- Compromised credentials check
- Phone and email verification
- Adaptive authentication



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

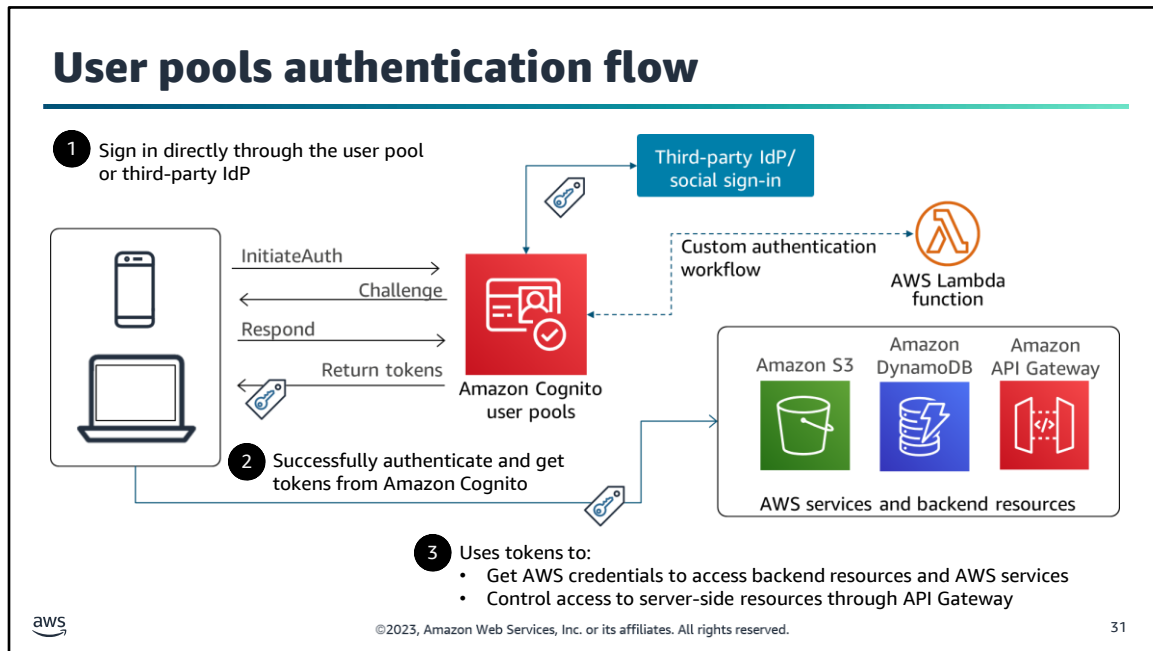
30

Amazon Cognito user pools provide several security features and functionality for user authentication. As mentioned before, Amazon Cognito offers sign-up and sign-in services, including social network sign-in with Facebook, Google, and Login with Amazon, as well as sign-in with SAML identity providers from your user pool. Amazon Cognito user pools come with a built-in, customizable web UI to sign in users, manage user directories, and access user profiles.

You can use an Amazon Cognito user pool to control who can access your API in API Gateway. API Gateway validates the tokens from a successful user pool authentication, and uses them to grant your users access to resources including AWS Lambda functions or your own API. You can also use groups in a user pool to control permissions with API Gateway by mapping group membership to IAM roles. The groups that a user is a member of are included in the ID token that a user pool provides when your application user signs in.

Security features such as checks for compromised credentials, phone and email verification, and adaptive authentication are also included. The compromised credentials feature protects your users' accounts by preventing them from reusing credentials (a user name and password pair) that have been exposed elsewhere. This feature addresses the issue of users reusing the same credentials for multiple websites and applications. With adaptive authentication you can configure your user pool to block suspicious sign-ins or add second factor authentication in response to an increased risk level.

A security best practice is to never use the same user name and password in different systems. If an attacker is able to obtain user credentials through a breach of one system, they could use those user credentials to access other systems. AWS has formed partnerships and programs so that Amazon Cognito is informed when a set of credentials has been compromised elsewhere.



A *user pool* is a user directory in Amazon Cognito. Your application users can sign in either directly through a user pool or federate through a third-party IdP.

To authenticate, a user answers successive challenges until authentication either fails or the user is issued tokens. With these two steps, which can be repeated to include different challenges, you can support any custom authentication flow.

You can customize your authentication flow with Lambda triggers. These triggers issue and verify their own challenges as part of the authentication flow. Challenges might include password verification, MFA enabled, CAPTCHA, or secret questions and answers.

You can also use a Lambda trigger to perform post-authentication tasks such as logging events for custom analytics. For more information, see Customizing User Pool Workflows with Lambda Triggers in the *Amazon Cognito Developer Guide* at <https://docs.aws.amazon.com/cognito/latest/developerguide/cognito-user-identity-pools-working-with-aws-lambda-triggers.html>.

The user pool manages the overhead of handling the tokens that are returned from social sign-in through Facebook, Google, and Login with Amazon, and from OIDC and SAML IdPs. Whether users sign in directly or through a third party, all members of the user pool have a directory profile, which you can access through an SDK. After a successful user pool sign-in, your web or mobile app will receive user pool tokens from Amazon Cognito.

You can use those tokens to retrieve AWS credentials that allow your application to access other AWS services, or you might choose to use them to control access to your server-side resources or to API Gateway. You can also create user pool groups to manage permissions and to represent different types of users.

For more information about the different use cases for Amazon Cognito user pools (including how to grant your users access to AWS AppSync resources with tokens from a successful Amazon Cognito authentication), see Common Amazon Cognito Scenarios at <https://docs.aws.amazon.com/cognito/latest/developerguide/cognito-scenarios.html>.

User pool tokens

Identity token

- **Authorizes API calls.** Contains claims about the **identity of the user** (for example, name and email).



aws

Signed

Access token

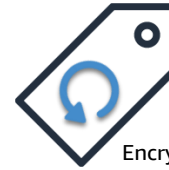
- **Authorizes API calls** based on the custom scopes of **specified access-protected resources.**



Signed

Refresh token

- Contains the information necessary **to obtain a new ID or access token.**



Encrypted

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

32

When you sign in by using a user pool, regardless of where the identity lives (in the pool or coming from a federated third-party IdP), Amazon Cognito gives you three JWTs:

- ID
- Access
- Refresh

ID and access tokens are signed, not encrypted. The refresh token is encrypted.

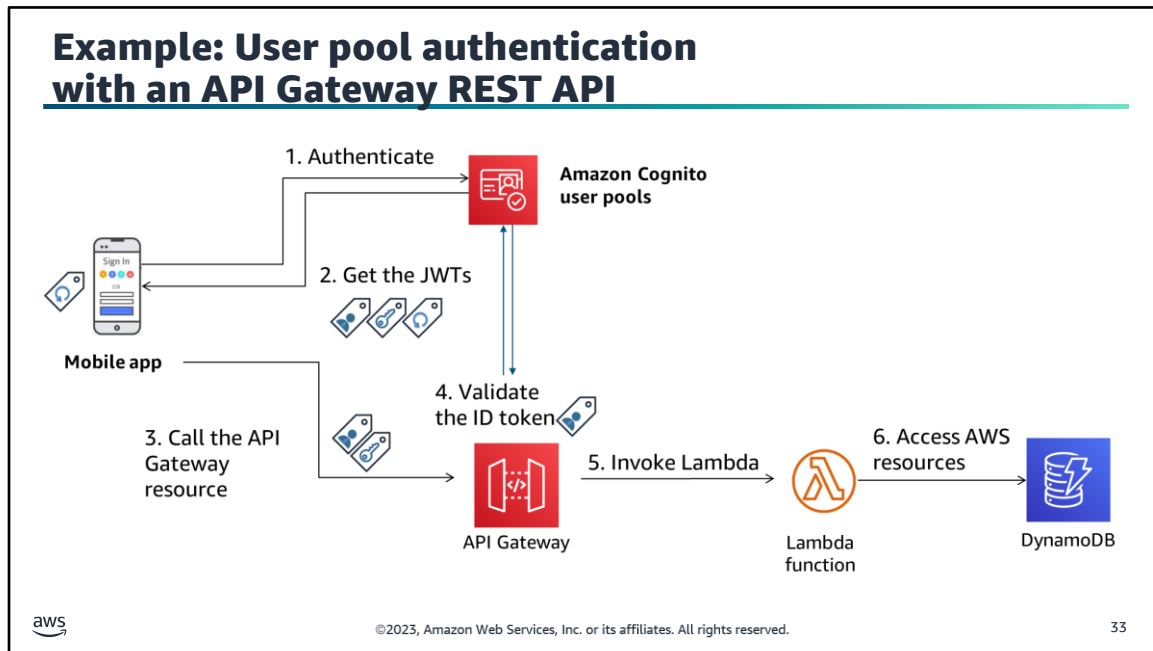
The developer's job is to take the ID and access tokens, and pass them along to authorize access to application resources.

The client must save the refresh token to silently refresh the ID and access tokens behind the scenes.

By design, tokens live for a relatively short time. You can choose how long your access and refresh tokens remain valid. You can configure access tokens to expire in as little as 5 minutes or as long as 24 hours. You can configure refresh tokens to expire in as little as 1 hour or as long as 10 years.

These tokens simplify identity for developers and provide the ability, in a lightweight manner, to pass user context to downstream services so that developers can easily get this user context.

When you use the user pool directory as your IdP, AWS hosts the entire experience. If you set up user pools to federate with a third-party IdP, Amazon Cognito user pools trusts those identities and redirects them as if the users were in its directory.



This diagram depicts a flow that uses Amazon Cognito user pools with API Gateway.

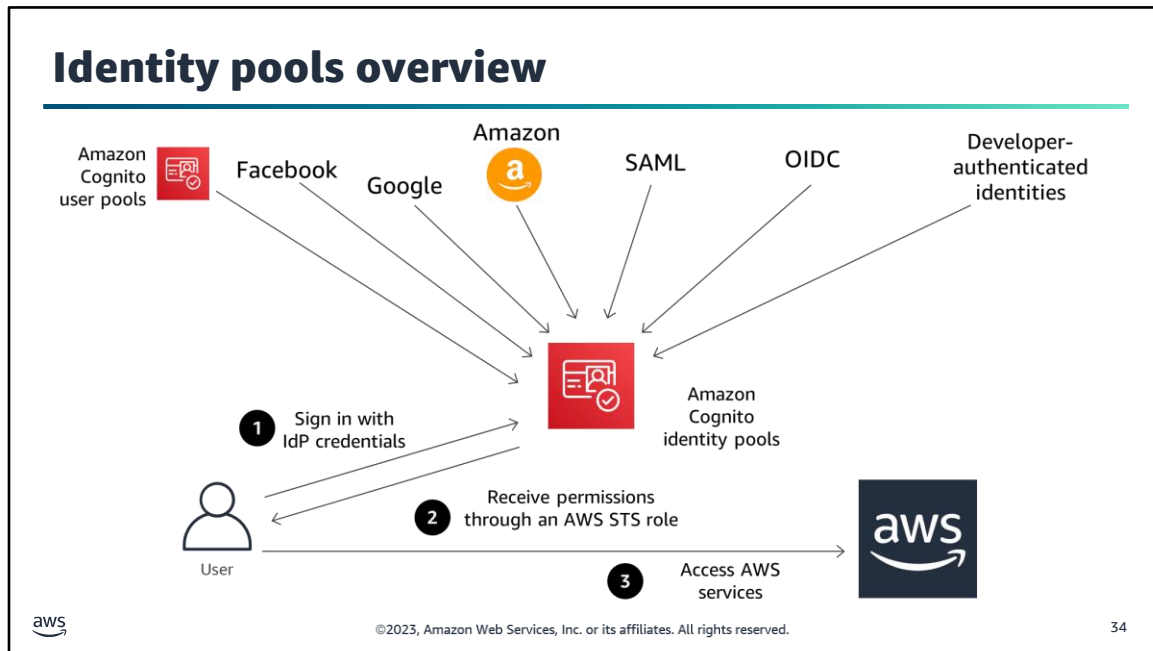
The client first authenticates with the user pool and gets the three JWTs in return.

The client then passes the ID and access token in the header as part of the call to API Gateway.

API Gateway validates the token before it invokes the resource that it integrates with on the backend (in this example, a Lambda function).

Depending on how you've written your API and application, API Gateway might pass on the ID token or the access token. If you don't need to further scope the access allowed, pass the ID token. If you need to scope access further, you can use the access token and configure predefined attributes within the API method request by using OAuth scopes.

For details about configuring these options, see [Integrate a REST API with an Amazon Cognito User Pool](https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-enable-cognito-user-pool.html) at <https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-enable-cognito-user-pool.html>.



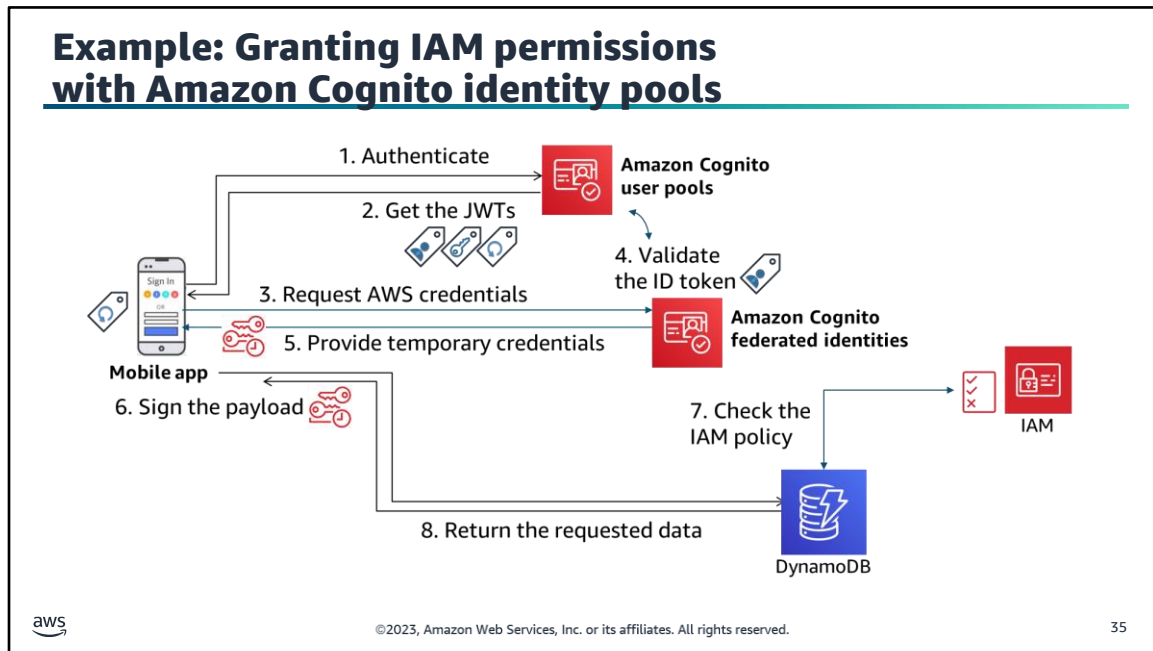
You can use web identity federation in your mobile apps to enable users to sign in using supported IdPs (for example, Login with Amazon, Facebook, or Google) and to trade an authentication token from these providers for temporary AWS security credentials. The advantage is that you can build mobile apps without writing any backend code to integrate with these IdPs. In addition, you don't need to embed long-term AWS credentials in your app.

With Amazon Cognito identity pools, you can create unique identities for your users and authenticate them with web IdPs. With an identity, you can obtain temporary, limited-privilege AWS credentials through AWS STS to access other AWS services. Amazon Cognito also supports developer-authenticated identities, which provides the ability to register and authenticate users through your own backend authentication process. This diagram presents the overall functionality of identity pools.

Users sign in by using the credentials of their IdP of choice. Amazon Cognito validates user credentials with the information received from the IdP. Amazon Cognito then replies with permissions to access AWS services through an AWS STS role. Notice that in this view, Amazon Cognito user pools is just one other IdP, similar to Facebook, Google, and others.

Amazon Cognito identity pools support the following IdPs:

- Amazon Cognito user pools
- Public IdPs, such as Login with Facebook, Google, or Amazon
- SAML IdPs
- OIDC IdPs
- Developer-provided authenticated identities



This example uses federated identities with IAM-based authorization to provide temporary IAM credentials.

The client gets JWTs from the Amazon Cognito user pool and uses those to request temporary AWS credentials. Amazon Cognito identity pools validates the ID token and then returns temporary credentials.

The client uses these credentials to sign the payload and call the service API.

The service (in this example, Amazon DynamoDB) checks the policy associated with the temporary credentials and allows the actions that the policy provides.

For more information, see Identity Pools (Federated Identities) Authentication Flow at <https://docs.aws.amazon.com/cognito/latest/developerguide/authentication-flow.html>.

Section 4 key takeaways



- Use **Amazon Cognito** to manage user access to AWS resources.
- **User pools** are directories of users who can access AWS resources.
- You associate users with credentials in **identity pools**.
- Authenticate access with Amazon Cognito by using user pools or **IdP credentials**.

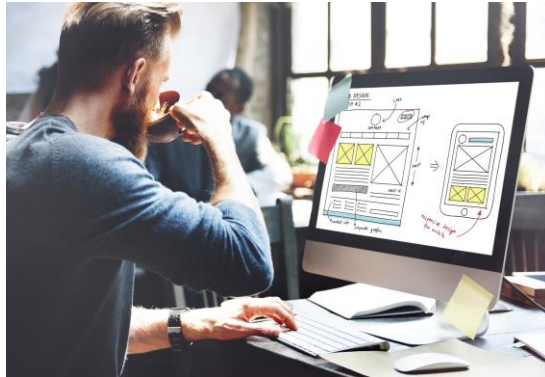
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

36

The following are the key takeaways from this section of the module:

- Use Amazon Cognito to manage user access to AWS resources.
- User pools are directories of users who can access AWS resources.
- You associate users with credentials in identity pools.
- Authenticate access with Amazon Cognito by using user pools or IdP credentials.

Lab 12.1: Implementing Application Authentication Using Amazon Cognito



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

37

You will now complete Lab 12.1: Implementing Application Authentication Using Amazon Cognito.

Lab: Scenario

In this lab, Sofía will use Amazon Cognito to integrate an authentication mechanism into the website. Frank will be able to log in to the website to confirm his identity before he requests the report. Then, she will connect the REST API endpoint to the café website so that he can make his report request directly from the site.



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

38

In this lab, you will integrate Amazon Cognito into the café website to provide authentication so that Frank can login to the website and request the inventory report.

Lab: Tasks

1. Preparing the development environment
2. Configuring a Cognito user pool and app client
3. Configuring the client app
4. Integrating the Amazon Cognito hosted URI into the website
5. Observing the REST API endpoint details and testing
6. Creating a user for the Amazon Cognito user pool
7. Configuring an API Gateway authorizer
8. Testing the request process from the website



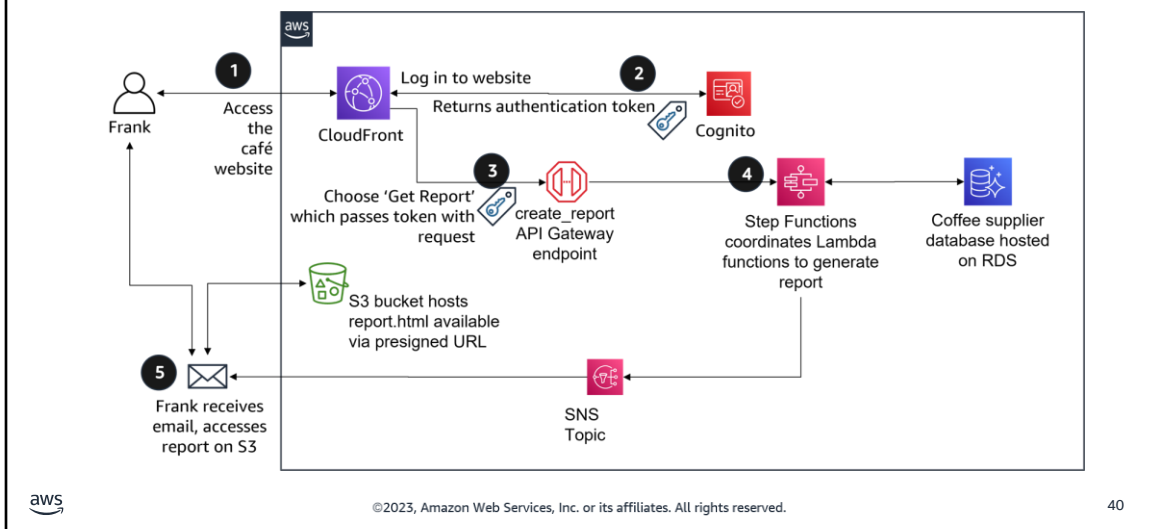
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

39

In this lab, you will complete the following tasks:

1. Preparing the development environment
2. Configuring a Cognito user pool and app client
3. Configuring the client app
4. Integrating the Amazon Cognito hosted URI into the website
5. Observing the REST API endpoint details and testing
6. Creating a user for the Amazon Cognito user pool
7. Configuring an API Gateway authorizer
8. Testing the request process from the website

Lab: Final product



The diagram summarizes what you will have built after you complete the lab.



**Begin Lab 12.1:
Implementing
Application
Authentication Using
Amazon Cognito**

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

41

It is now time to start the lab.

Lab debrief: Key takeaways



aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

42

After you complete the lab, your educator might choose to lead a conversation about the key takeaways from the lab.



It's now time to review the module and wrap up with a knowledge check and discussion of a practice certification exam question.

Module summary

In summary, in this module, you learned how to do the following:

- List two aspects of securing applications
- Recall how to authenticate with AWS STS
- Describe how to build secure applications with Amazon Cognito
- Secure part of a website with Amazon Cognito



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

44

In summary, in this module, you learned how to do the following:

- List two aspects of securing applications
- Recall how to authenticate with AWS STS
- Describe how to build secure applications with Amazon Cognito
- Secure part of a website with Amazon Cognito

Complete the knowledge check



aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

45

It is now time to complete the knowledge check for this module.

Sample exam question

A developer is adding sign-up and sign-in functionality to an application. The application is required to make an API call to a custom analytics solution to log user sign-in events.

Which combination of actions should the developer take to satisfy these requirements? (Select TWO.)

Identify the key words and phrases before continuing.

The following are the key words and phrases:

- Sign-up and sign-in functionality
- Custom analytics solution
- Sign-in events



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

46

It is important to fully understand the scenario and question being asked before even reading the answer choices. Find the keywords in this scenario and question that will help you find the correct answer.

Sample exam question: Response choices

A developer is adding **sign-up and sign-in functionality** to an application. The application is required to make an API call to a **custom analytics solution** to log user **sign-in events**.

Which combination of actions should the developer take to satisfy these requirements? (Select TWO.)

Choice	Response
A	Use Amazon Cognito to provide the sign-up and sign-in functionality.
B	Use IAM to provide the sign-up and sign-in functionality.
C	Invoke an API Gateway method to make the API call that the post-authentication event initiates.
D	Store the database credentials in AWS Secrets Manager. Configure Secrets Manager to automatically rotate the credentials.
E	Invoke a Lambda function to make the API call that the post-authentication event initiates.

 ©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved. 47

Now that we have bolded the keywords in this scenario, let us look at the answers.

Sample exam question: Answer

The correct answers are A and E.

Choice	Response
A	Use Amazon Cognito to provide the sign-up and sign-in functionality.
B	Use IAM to provide the sign-up and sign-in functionality.
C	Invoke an API Gateway method to make the API call that the post-authentication event initiates.
D	Store the database credentials in AWS Secrets Manager. Configure Secrets Manager to automatically rotate the credentials.
E	Invoke a Lambda function to make the API call that the post-authentication event initiates.

 ©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved. 48

Look at the answer choices and rule them out based on the keywords that were previously highlighted.

The correct answers are **A and E. Use Amazon Cognito to provide the sign-up and sign-in functionality and invoke a Lambda function to make the API call that the post-authentication event initiates.**

Amazon Cognito provides user sign-up, sign-in, and access control to web and mobile applications quickly and easily. Users can also create a Lambda function to make an API call to a custom analytics solution and then invoke that function with an Amazon Cognito post-authentication trigger.

Additional resources

- Blog posts
 - [Amazon Cognito Groups and Fine-Grained Role-Based Access Control](#)
 - [Use AWS Secrets Manager to Simplify the Management of Private Certificates](#)
 - [Using ACM Private Certificate Authority in a Multi-Account Environment by Using IAM Roles](#)
- Disaster Recovery on AWS workshop
 - [AWS Security Token Service](#)

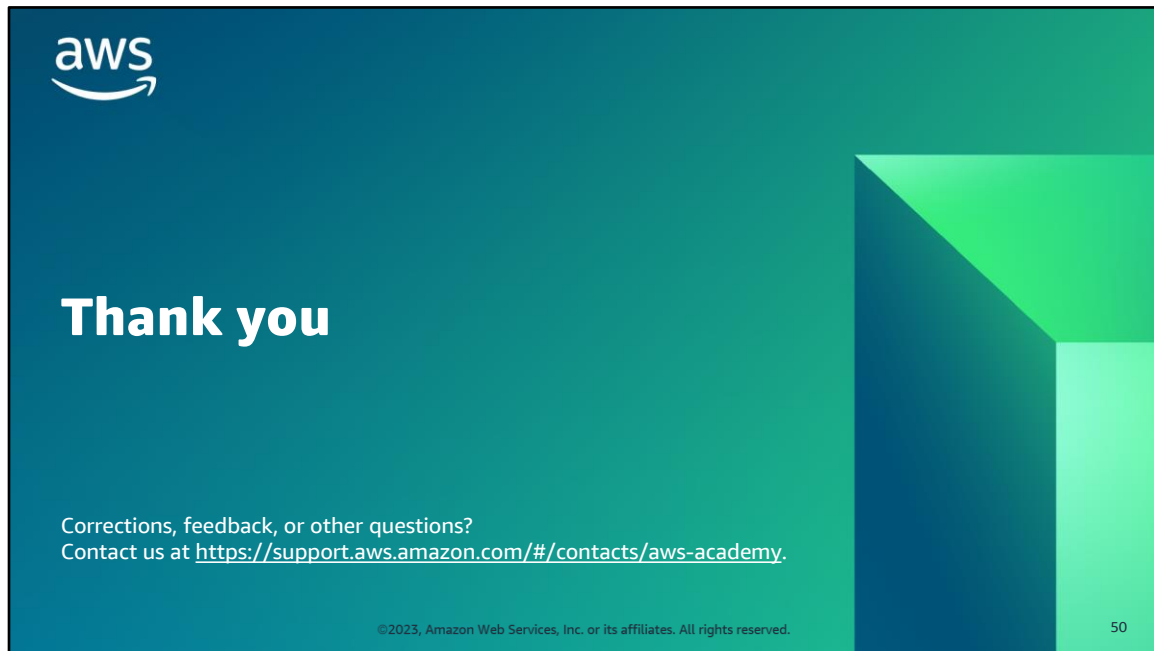


©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

49

To learn more about the topics covered in this module, you might find the following additional resources helpful:

- Blog posts:
 - Amazon Cognito Groups and Fine-Grained Role-Based Access Control: <https://aws.amazon.com/blogs/aws/new-amazon-cognito-groups-and-fine-grained-role-based-access-control-2/>.
 - Use AWS Secrets Manager to Simplify the Management of Private Certificates: <https://aws.amazon.com/blogs/security/use-aws-secrets-manager-to-simplify-the-management-of-private-certificates/>.
 - Using ACM Private Certificate Authority in a Multi-Account Environment by Using IAM Roles: <https://aws.amazon.com/blogs/security/using-acm-private-certificate-authority-multi-account-environment-using-iam-roles/>.
- Disaster Recovery on AWS workshop
 - AWS Security Token Service: <https://disaster-recovery.workshop.aws/en/services/security/iam/sts.html>.



Thank you for completing this module.